

## Unidad 6: Manejo de grandes volúmenes de datos

### Clase 12 – Big Data y visión sistémica de la ingeniería de datos

#### Objetivo

*Comprender el rol del Big Data y del ecosistema Hadoop en arquitecturas modernas de datos, analizando la integración de estructuras NoSQL con Data Marts y comparando su rendimiento frente a soluciones relacionales.*

#### Tema

Manejo de grandes volúmenes de datos.

Big Data: concepto y contexto.

Hadoop como ecosistema (HDFS, procesamiento distribuido).

### Big Data y ecosistema Hadoop en la ingeniería de datos moderna

#### Introducción conceptual

Big Data representa uno de los cambios más relevantes en la forma de diseñar, almacenar, procesar y explotar datos dentro de las organizaciones. No se trata solamente de “tener muchos datos”, sino de enfrentar escenarios donde la cantidad, la velocidad de generación, la diversidad de formatos y la necesidad de obtener valor superan las posibilidades prácticas de una arquitectura tradicional basada exclusivamente en bases de datos relacionales.

Una organización puede administrar correctamente sus ventas, clientes, productos y facturas en una base relacional como SQL Server, Oracle o PostgreSQL. Sin embargo, cuando además necesita conservar millones de eventos de navegación, registros de sensores, logs de aplicaciones, documentos, archivos externos, interacciones digitales, ubicaciones geográficas, búsquedas, clics y comportamientos históricos, el problema deja de ser solamente transaccional. Aparece una necesidad más amplia: capturar datos diversos, almacenarlos a gran escala, procesarlos en forma eficiente y transformarlos en información útil para la toma de decisiones.

En ese contexto surge Hadoop como una de las respuestas tecnológicas más importantes de la historia del Big Data. Hadoop no debe entenderse como una base de datos, ni como una herramienta aislada, sino como un ecosistema de almacenamiento y procesamiento distribuido. Su aporte central fue permitir que grandes volúmenes de datos pudieran dividirse, distribuirse y procesarse en clústeres de múltiples nodos, acercando el procesamiento al lugar donde se encuentran los datos. Apache Hadoop organiza su núcleo en Hadoop Common, Hadoop Distributed File System, Hadoop YARN y Hadoop MapReduce. HDFS ofrece acceso de alto rendimiento a datos distribuidos;

YARN gestiona recursos y planificación de trabajos; MapReduce permite procesamiento paralelo de grandes conjuntos de datos.

El valor profesional de Big Data y Hadoop no está en acumular tecnología, sino en resolver problemas concretos: reducir tiempos de procesamiento, almacenar información histórica de manera escalable, integrar datos estructurados y no estructurados, generar insumos para analítica avanzada, alimentar modelos predictivos, construir indicadores de negocio y fortalecer procesos de auditoría, control, logística, marketing, operaciones o experiencia del cliente.

### **Marco conceptual principal**

#### **Big Data como problema de arquitectura y negocio**

Big Data puede definirse como un conjunto de enfoques, arquitecturas, tecnologías y prácticas orientadas a trabajar con datos cuyo volumen, velocidad, variedad o complejidad exceden las capacidades tradicionales de almacenamiento, procesamiento y análisis. La clave no es el tamaño en sí mismo, sino la relación entre los datos disponibles, la capacidad técnica para procesarlos y el valor que se desea obtener.

Las dimensiones más utilizadas para comprender Big Data son las llamadas “V”:

**Volumen:** refiere a la cantidad de datos generados y acumulados. Puede tratarse de millones de transacciones, registros históricos, eventos de usuarios, archivos, logs o datos provenientes de dispositivos.

**Velocidad:** expresa la rapidez con la que los datos se generan, ingresan, cambian o deben ser procesados. En algunos casos se trabaja por lotes; en otros, cerca del tiempo real.

**Variedad:** indica la diversidad de formatos. Los datos pueden estar en tablas relacionales, archivos CSV —Comma-Separated Values, valores separados por coma—, documentos JSON —JavaScript Object Notation—, logs, imágenes, audios, textos, coordenadas, sensores o documentos.

**Veracidad:** se relaciona con la confiabilidad, calidad, consistencia y trazabilidad de los datos. Un gran volumen de datos sin control de calidad puede producir análisis erróneos.

**Valor:** es la dimensión más importante desde el punto de vista profesional. Los datos solo justifican su tratamiento si permiten mejorar decisiones, reducir riesgos, optimizar procesos, descubrir patrones o generar nuevos servicios.

Big Data, por lo tanto, no debe confundirse con una moda tecnológica. Una organización puede tener muchos datos y no tener una estrategia Big Data. También puede tener menos volumen, pero alta variedad, alta velocidad o una necesidad analítica compleja que justifique una arquitectura diferente. El criterio profesional no es “cuántos datos existen”, sino qué problema se necesita resolver, qué capacidad técnica se requiere y qué valor concreto puede producirse.

### Limitaciones de las arquitecturas tradicionales

Las bases de datos relacionales siguen siendo fundamentales. Son adecuadas para registrar operaciones transaccionales, garantizar integridad, controlar consistencia, modelar entidades de negocio y ejecutar consultas estructuradas mediante SQL —Structured Query Language, lenguaje de consulta estructurado—. Sin embargo, no siempre son la mejor opción para conservar grandes volúmenes de datos crudos, procesar archivos masivos, integrar datos semiestructurados o ejecutar cargas analíticas distribuidas.

El problema aparece cuando una única base o un único servidor empieza a concentrar demasiadas responsabilidades: registrar operaciones, almacenar histórico, procesar información masiva, responder consultas operativas, alimentar reportes, sostener integraciones y ejecutar análisis complejos. Esa concentración puede generar altos costos, baja performance, dificultad de escalabilidad y dependencia excesiva de una única plataforma.

Las arquitecturas modernas tienden a distribuir responsabilidades. Una base relacional puede seguir cumpliendo el rol transaccional; una base NoSQL —Not Only SQL, no solamente SQL— puede administrar documentos, grafos, claves-valor o columnas distribuidas; un repositorio de datos masivos puede conservar información cruda; un motor distribuido puede procesar grandes volúmenes; y una capa analítica puede transformar los resultados en indicadores, tableros o modelos de decisión.

### Hadoop como respuesta al desafío Big Data

Hadoop surge como una forma de resolver dos necesidades centrales: almacenar grandes volúmenes de datos en forma distribuida y procesarlos paralelamente. Su principio arquitectónico es simple pero poderoso: en lugar de mover todos los datos hacia una única máquina central, se distribuyen los datos entre distintos nodos y se ejecuta el procesamiento cerca de donde esos datos están ubicados.

El Hadoop Distributed File System —HDFS, sistema de archivos distribuido de Hadoop— está diseñado para ejecutarse sobre hardware común, tolerar fallos, ofrecer alto rendimiento de acceso a datos y trabajar con archivos grandes. La documentación oficial de HDFS enfatiza que fue pensado para grandes conjuntos de datos, acceso secuencial de alto rendimiento y procesamiento por lotes más que para uso interactivo de baja latencia. También plantea una idea central de diseño: mover la computación suele ser más eficiente que mover grandes volúmenes de datos.

HDFS organiza los archivos en bloques distribuidos entre nodos. El NameNode administra la metadata del sistema de archivos, es decir, mantiene información sobre nombres, directorios, bloques y ubicación lógica. Los DataNodes almacenan los bloques reales de datos y atienden operaciones de lectura y escritura. Esta arquitectura permite que el sistema distribuya almacenamiento, replique bloques y sostenga disponibilidad frente a fallas de nodos.

Hadoop YARN —Yet Another Resource Negotiator, otro negociador de recursos— cumple el rol de administrador de recursos del clúster. Su función es asignar capacidad de procesamiento, memoria y ejecución a las aplicaciones que corren sobre Hadoop. Esto permite que el ecosistema no dependa únicamente de MapReduce, sino que pueda ejecutar distintos motores y trabajos sobre una infraestructura compartida.

Hadoop MapReduce es el modelo clásico de procesamiento distribuido. Divide una tarea en dos grandes etapas: Map y Reduce. La etapa Map procesa fragmentos de datos y genera pares clave-valor; la etapa Reduce agrupa, consolida y produce resultados finales. Aunque hoy existen motores más flexibles y rápidos para muchos escenarios, MapReduce sigue siendo fundamental para comprender la lógica del procesamiento distribuido: dividir el trabajo, ejecutarlo en paralelo y consolidar resultados.

### **Hadoop no es una base de datos**

Una confusión frecuente es considerar Hadoop como una base de datos. Hadoop no reemplaza directamente a SQL Server ni cumple la misma función que una base transaccional. Su naturaleza es diferente: se orienta al almacenamiento distribuido de archivos y al procesamiento de grandes volúmenes de datos. Puede formar parte de una arquitectura analítica, pero no es la herramienta ideal para operaciones transaccionales de baja latencia, actualizaciones frecuentes registro por registro o consultas operativas inmediatas.

La diferencia profesional es importante. Una base relacional responde muy bien preguntas como: “¿Cuál es el saldo actual de este cliente?”, “¿Qué factura se emitió?”, “¿Qué pedido debe despacharse?”. Hadoop responde mejor a escenarios como: “¿Qué patrones aparecen en tres años de navegación?”, “¿Qué productos generan más abandono de carrito?”, “¿Qué logs muestran comportamientos anómalos?”, “¿Qué registros históricos deben procesarse para alimentar un modelo de riesgo?”.

### **El ecosistema Hadoop ampliado**

Con el tiempo, Hadoop dejó de ser solamente HDFS, YARN y MapReduce. Alrededor de su núcleo se consolidó un ecosistema de herramientas para consultar, procesar, administrar, integrar y explotar datos.

Apache Hive es una pieza clave porque acerca Hadoop al mundo analítico conocido por quienes trabajan con SQL. Hive permite leer, escribir y administrar grandes volúmenes de datos ubicados en almacenamiento distribuido usando SQL. La documentación oficial lo define como un sistema de data warehouse distribuido y tolerante a fallos, orientado a analítica a escala masiva sobre datos residentes en almacenamiento distribuido.

Apache Spark, por su parte, representa una evolución muy utilizada para procesamiento distribuido moderno. Spark permite ejecutar tareas de ingeniería de datos, ciencia de datos y machine learning —aprendizaje automático— en una sola

máquina o en clústeres. También integra procesamiento batch, streaming, consultas SQL, Python, Scala, Java y R, lo que lo vuelve más flexible para proyectos actuales.

Apache HBase puede ubicarse como una base NoSQL distribuida asociada históricamente al ecosistema Hadoop, útil para grandes volúmenes de datos dispersos y acceso por clave. Apache Kafka suele aparecer en arquitecturas Big Data como plataforma de eventos y streaming, especialmente cuando se requiere capturar información en movimiento. Apache Oozie, Apache Sqoop, Apache Flume y otras herramientas han sido utilizadas para orquestación, integración e ingesta, aunque algunas han perdido centralidad frente a soluciones más modernas.

El punto relevante es comprender Hadoop como ecosistema. En una arquitectura real, rara vez se habla de una sola herramienta. Se combinan almacenamiento distribuido, procesamiento paralelo, consulta SQL, ingesta de datos, orquestación, seguridad, monitoreo, gobierno de datos y explotación analítica.

### **Cómo se accede y se utiliza Hadoop**

Hadoop puede utilizarse desde distintos niveles técnicos. Un perfil de infraestructura o ingeniería puede acceder por consola mediante comandos HDFS para crear directorios, subir archivos, listar contenido, consultar rutas, revisar permisos o recuperar resultados. Un perfil de datos puede utilizar Hive para consultar datasets distribuidos mediante una sintaxis similar a SQL. Un perfil de ingeniería avanzada puede trabajar con Spark, notebooks, scripts en Python, Java o Scala, flujos programados y procesos de transformación.

También puede accederse mediante servicios administrados en la nube. Esto es especialmente importante para una mirada actual: muchas organizaciones ya no instalan clústeres Hadoop manualmente en servidores propios, sino que consumen servicios gestionados. Microsoft Azure HDInsight permite ejecutar frameworks abiertos como Apache Hadoop, Spark, Hive y Kafka en un servicio empresarial administrado. Google Cloud integró Dataproc y sus servicios serverless bajo Managed Service for Apache Spark, permitiendo ejecutar Spark en modalidad serverless o con clústeres administrados, y también soporta herramientas abiertas como Hadoop, Flink y Trino.

Amazon EMR —Elastic MapReduce— también forma parte de este tipo de servicios administrados. Su documentación presenta Hive como un paquete open source de data warehouse y analítica que corre sobre un clúster Hadoop y permite usar HiveQL —Hive Query Language—, un lenguaje similar a SQL, evitando escribir programas de menor nivel para ciertos trabajos analíticos.

Desde el punto de vista profesional, esta variedad de accesos significa que Hadoop no es solo una tecnología para administradores de servidores. Puede intervenir en pipelines de datos, procesos ETL —Extract, Transform, Load: extraer, transformar y cargar—, procesos ELT —Extract, Load, Transform: extraer, cargar y transformar—, data lakes, preparación de datasets, reporting, auditoría continua, machine learning y análisis histórico.

### Decisión empresarial de adopción

La decisión de incorporar Hadoop o tecnologías relacionadas no debería tomarse por moda, sino por necesidad técnica y valor de negocio. Una empresa evalúa este tipo de soluciones cuando enfrenta alguno de estos escenarios:

- necesidad de almacenar datos históricos masivos a menor costo relativo;
- procesos batch que tardan demasiado en una arquitectura centralizada;
- datos provenientes de muchas fuentes y formatos;
- requerimiento de conservar datos crudos para análisis futuros;
- necesidad de preparar datasets para modelos analíticos o predictivos;
- crecimiento sostenido de logs, eventos, sensores o interacciones digitales;
- integración de datos operativos, semiestructurados y no estructurados;
- necesidad de escalar procesamiento sin depender de una única máquina.

La adopción responsable exige analizar costo total de propiedad, capacidades del equipo, seguridad, gobierno de datos, madurez de la organización, criticidad de los procesos, necesidades de latencia y alternativas disponibles. Hadoop puede aportar mucho valor, pero también requiere administración, monitoreo, diseño de particiones, control de permisos, políticas de retención, calidad de datos y una arquitectura clara. Incorporarlo sin un caso de uso sólido puede generar complejidad innecesaria.

### Aplicación profesional

En Ingeniería de Datos II, Big Data y Hadoop permiten comprender cómo se amplía el diseño de persistencia más allá de la base de datos individual. El foco profesional se desplaza desde “qué tabla o colección uso” hacia “qué arquitectura de datos necesito para resolver el problema completo”.

Una solución moderna puede integrar diferentes capas. La capa transaccional registra operaciones oficiales. La capa documental conserva información flexible. La capa clave-valor administra estados temporales. La capa de grafos representa relaciones. La capa distribuida conserva históricos masivos. La capa de procesamiento transforma datos crudos. La capa analítica organiza resultados para indicadores, tableros, modelos o decisiones.

Por ejemplo, en una plataforma de comercio electrónico, SQL Server podría almacenar clientes, pedidos, pagos y facturas. MongoDB podría conservar sesiones de navegación con atributos variables. Redis podría manejar carritos activos o recomendaciones temporales. Cassandra podría guardar eventos masivos por usuario, fecha y región. Neo4j podría representar relaciones entre usuarios, productos, categorías y similitudes. Hadoop o un data lake podrían conservar el histórico completo de eventos crudos. Spark o Hive podrían procesar esos datos para generar métricas de conversión,



abandono, recurrencia, segmentación o recomendación. Finalmente, un Data Mart podría exponer indicadores para Business Intelligence —BI, inteligencia de negocios—.

El valor profesional está en asignar correctamente responsabilidades. No todos los datos necesitan la misma estructura, la misma consistencia, la misma latencia ni el mismo motor. La ingeniería de datos moderna exige comprender el ciclo completo: origen, ingesta, almacenamiento, procesamiento, calidad, integración, explotación y gobierno.

También resulta relevante para auditoría, riesgos y control interno. Un área de auditoría continua puede necesitar procesar millones de registros de cuentas por pagar, accesos de usuarios, cambios maestros, logs de sistemas, excepciones operativas o transacciones históricas. Una base relacional puede servir para la operación diaria, pero el análisis masivo de patrones, duplicidades, combinaciones anómalas o eventos históricos puede requerir procesamiento distribuido. En ese contexto, Hadoop o tecnologías equivalentes pueden funcionar como capa de almacenamiento y procesamiento para detectar excepciones, preparar evidencias y alimentar tableros de riesgo.

### **Buenas prácticas**

La primera buena práctica es partir del problema, no de la herramienta. Hadoop tiene sentido cuando existe una necesidad concreta de almacenamiento distribuido, procesamiento masivo, análisis histórico o integración de datos diversos. Si el problema puede resolverse de manera simple con una base relacional, una consulta optimizada o un Data Mart pequeño, incorporar Hadoop puede aumentar la complejidad sin aportar valor proporcional.

La segunda buena práctica es separar claramente las capas de la arquitectura. Los datos crudos no deberían mezclarse con datos refinados sin control. Una arquitectura madura distingue entre zona de ingesta, zona cruda, zona procesada, zona curada y zona analítica. Esta separación permite trazabilidad, control de calidad y reutilización.

La tercera buena práctica es diseñar pensando en el patrón de acceso. No alcanza con almacenar datos: hay que saber cómo se consultarán, con qué frecuencia, por qué claves, con qué granularidad y para qué propósito. En Big Data, una mala organización física o lógica puede volver costoso cualquier procesamiento posterior.

La cuarta buena práctica es controlar calidad y gobierno de datos desde el inicio. Big Data no significa aceptar desorden. Es necesario definir reglas de validación, diccionarios de datos, linaje, responsables, permisos, políticas de retención, cifrado y monitoreo. Cuanto mayor es el volumen, mayor es el impacto de errores repetidos.

La quinta buena práctica es evitar la dependencia excesiva de una sola tecnología. Hadoop es valioso como base conceptual y como ecosistema, pero muchas arquitecturas actuales combinan HDFS, almacenamiento de objetos, Spark, Hive, Kafka, bases NoSQL, servicios cloud y herramientas de orquestación. El diseño debe ser flexible y responder al caso de uso.

La sexta buena práctica es evaluar el costo operativo. Un clúster propio requiere administración, monitoreo, seguridad, actualizaciones, capacidad técnica y soporte. Los servicios administrados reducen parte de esa carga, pero introducen costos de consumo, dependencia de proveedor y necesidad de controlar escalabilidad, permisos y gobierno.

La séptima buena práctica es construir valor incremental. No conviene iniciar con una arquitectura sobredimensionada. Un enfoque profesional comienza con un caso de uso concreto, mide beneficios, valida resultados y luego escala. La pregunta clave no es “qué tecnología se puede usar”, sino “qué decisión mejora gracias a esta arquitectura”.

### Ejemplos aplicados

#### Análisis de comportamiento en una plataforma de comercio electrónico

Una empresa de comercio electrónico registra correctamente sus ventas en una base relacional. Cada pedido confirmado queda asociado a un cliente, un producto, una fecha, un medio de pago y una factura. Ese modelo transaccional funciona bien para operar el negocio.

El problema aparece cuando la empresa quiere comprender por qué muchos usuarios abandonan el carrito antes de comprar. Para responder esa pregunta no alcanza con mirar ventas confirmadas. Es necesario analizar búsquedas, clics, productos vistos, tiempo de permanencia, dispositivo utilizado, origen de la visita, campañas de marketing y eventos de abandono. Esos datos pueden generarse en millones de registros diarios y tener formatos variables.

Una arquitectura Big Data permitiría almacenar los eventos crudos en un repositorio distribuido, procesarlos por lotes para calcular patrones de comportamiento y luego generar indicadores como tasa de conversión, productos con mayor abandono, categorías con mayor interés no convertido, horarios de mayor navegación y segmentos de usuarios con intención de compra. Hadoop podría intervenir como capa de almacenamiento distribuido mediante HDFS y como base conceptual de procesamiento distribuido; Spark o Hive podrían usarse para transformar los eventos en datasets analíticos. El resultado final podría alimentar un Data Mart comercial con Key Performance Indicators —KPI, indicadores clave de desempeño— para marketing, ventas y experiencia de usuario.

El valor profesional no está en guardar más datos, sino en transformar eventos dispersos en decisiones: rediseñar la experiencia de compra, ajustar promociones, mejorar recomendaciones o detectar problemas en el proceso de pago.

#### Auditoría continua sobre operaciones de cuentas por pagar

Una organización multinacional tiene múltiples sistemas de gestión, proveedores, facturas, órdenes de compra, pagos y aprobaciones. La información transaccional está distribuida entre sistemas Enterprise Resource Planning —ERP, planificación de recursos empresariales— y bases relacionales. El área de auditoría necesita detectar



pagos duplicados, proveedores con datos bancarios modificados recientemente, aprobaciones fuera de política, facturas cargadas en fechas inusuales y combinaciones anómalas entre usuario, proveedor, monto y centro de costo.

Si el volumen histórico es bajo, puede bastar con consultas SQL y reportes tradicionales. Pero si se necesitan analizar años de operaciones de varias compañías, logs de cambios maestros, archivos exportados, evidencias documentales y eventos de aprobación, una arquitectura Big Data puede ser más adecuada. Los datos crudos pueden almacenarse en una capa distribuida; luego se procesan para normalizar formatos, detectar duplicidades, cruzar fuentes y generar alertas. Los resultados refinados se publican en un modelo analítico para seguimiento de hallazgos, tableros de riesgo y revisión de excepciones.

En este caso, Hadoop o tecnologías equivalentes aportan capacidad para conservar historia, procesar volumen y sostener trazabilidad. La base relacional continúa siendo relevante para la operación oficial, pero el análisis masivo se desplaza hacia una arquitectura preparada para exploración, procesamiento distribuido y generación de evidencia.

### **Monitoreo de sensores en una operación logística**

Una empresa logística equipa sus vehículos con dispositivos de Internet of Things —IoT, Internet de las Cosas— que envían datos de ubicación, temperatura, velocidad, paradas, consumo y estado del recorrido. Los datos llegan constantemente y pueden acumularse en grandes volúmenes. La empresa no solo quiere saber dónde está un camión en un momento puntual, sino analizar patrones históricos: demoras frecuentes, rutas ineficientes, zonas de congestión, desvíos, alertas de temperatura y cumplimiento de tiempos de entrega.

Una arquitectura tradicional podría registrar entregas y viajes finalizados, pero no sería suficiente para explotar todo el flujo de eventos. Una solución Big Data permitiría almacenar los registros históricos, procesarlos por período, región o unidad de transporte, y generar modelos de análisis para optimización logística. Hadoop puede servir para comprender el almacenamiento distribuido de grandes volúmenes históricos; Spark puede procesar los datos por ventanas de tiempo; y una capa de BI puede exponer indicadores de puntualidad, eficiencia, riesgo operativo y cumplimiento de servicio.

El beneficio profesional es directo: reducción de costos, mejora de planificación, detección temprana de desvíos y mayor capacidad de control.

### **Síntesis conceptual final**

Big Data no es una tecnología aislada ni una simple acumulación de datos. Es una respuesta arquitectónica y profesional a escenarios donde los datos crecen en volumen, velocidad, variedad y complejidad, y donde las organizaciones necesitan convertir esa información en valor. Su importancia radica en permitir nuevas formas de análisis, control, predicción, optimización y toma de decisiones.

Hadoop ocupa un lugar fundamental dentro de esta evolución porque introdujo una forma práctica de pensar el almacenamiento y el procesamiento distribuido. HDFS permitió distribuir archivos grandes entre nodos; YARN organizó recursos del clúster; MapReduce ofreció un modelo de procesamiento paralelo; y el ecosistema ampliado incorporó herramientas como Hive, Spark, HBase y servicios administrados en la nube. Aunque muchas arquitecturas actuales utilizan motores más modernos, servicios cloud o enfoques lakehouse, los principios de Hadoop siguen siendo esenciales para comprender la ingeniería de datos contemporánea.

La aplicación profesional exige criterio. No todos los problemas requieren Hadoop ni toda arquitectura Big Data debe ser compleja. La decisión correcta depende del caso de uso, del volumen, de la frecuencia de procesamiento, del tipo de dato, de la necesidad de integración, del costo, de la gobernanza y del valor esperado. La ingeniería de datos aporta precisamente esa mirada: diseñar arquitecturas que conecten datos, tecnología y decisión de negocio de manera coherente, escalable y sustentable.